

Introduction à la programmation de bas niveau

L'architecture Intel x86

BUT Informatique à Arles

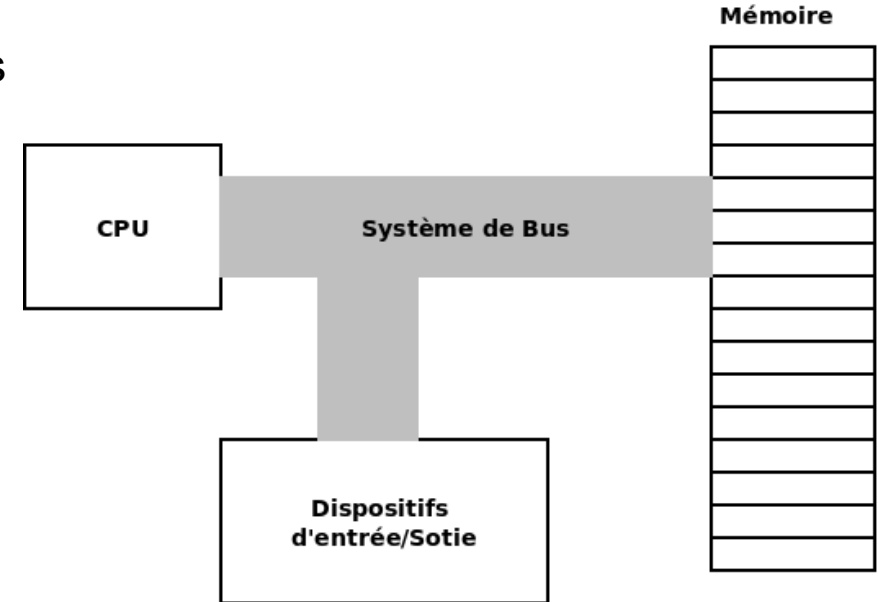
Ricardo Uribe Lobello

L'architecture Von Neumann

C'est l'architecture de base des ordinateurs actuels (avant les ordinateurs quantiques) Elle possède :

- L'unité central de traitement (Central Processing Unit) ;
- La mémoire ;
- Les dispositifs d'entrée/sortie.

Ces composants sont connectés par des systèmes de BUS.



L'architecture Von Neumann :

Le bus

- La CPU transmet des adresses en mémoire via les bus afin de stocker ou accéder aux informations dans l'adresse fournie.
- Les adresses des ports et pour les dispositifs d'entrée/sortie sont traitées comme des adresses en mémoire.
- Chacune de ces adresses sont uniques dans un segment de mémoire.
- En général, les données (ou adresses) sont mises dans le bus par de blocs de 64 bits (mais pas tous les bits au même temps) :
 - **Peripheral Component Interconnect Express (PCIe)** : transmet les informations de manière séquentielle. Par exemple, PCIe à 4, 8 ou 16 lignes mais à des grandes vitesses.
- Le circuit de contrôle du bus utilise des flags qui permettent de connaître la direction dans laquelle sont transmis les données. Dans le sens de la mémoire (requête du CPU) ou à partir de la mémoire (données à transmettre à la CPU).

L'architecture des processeurs Intel 80x86

Un peu d'histoire :

- **Années 70s** : La famille de processeurs **80x86** a été inaugurée dans les années 70s avec le processeur **8080** :
 - Processeur à 8 bits ;
 - Il possédait un bus d'adresses de 16 bits.
- **Années 80s** : Apparition des processeurs **8086** et **8088** :
 - Processeurs à 16 bits ;
 - Bus d'adressage de 20 bits.

Attention : tous les processeurs de cette famille partagent le même jeu d'instructions.

L'architecture Intel 80x86

Tous les processeurs 8080, 8086 et 8088 n'opéraient qu'en **mode réel** :

- **Mode réel** : Il s'agit des processeurs qui pouvaient accéder à n'importe quelle adresse de la mémoire, même celle qui est affectée aux autres programmes.

De plus, comment pouvoir accéder aux adresses de mémoire au delà de ce qui permettaient les 16 bits des enregistrements internes ?

- **Solution** : Au 16 bits des enregistrements, il faut ajouter 4 bits supplémentaires qui seront appelés des **registres de segment**. Ces 4 enregistrements sont CS (code segment), DS (Data segment), SS (Stack segment) et ES (Extra segment).
- **Conséquence** : les systèmes d'exploitation étaient mono-tâche et mono-utilisateur.

Très important : une bonne partie de la complexité des processeurs Intel vient de l'adoption de cette solution.

Le processeur Intel 80x286

L'apparition du processeur **80286** apporte les changements suivants :

- Les enregistrements restent à 16 bits ;
- Le bus d'adresses passe à 24 bits. Il est possible d'accéder jusqu'à 16 Mo adresses de la mémoire ;
- Quelques nouvelles instructions sont ajoutées au langage machine de base ;
- **Mode protégé** : C'est le premier processeur à ne pas permettre au programmeur d'assembleur l'accès au espace d'adressage d'autres programmes ;

Ce mode protégé n'a pas été très utilisé mais ce processeur fournissait déjà la mémoire virtuelle, des droits d'accès pour la sécurité et des niveaux de privilège pour l'exécution.

Le processeur Intel 80x386

Le processeur **80386** apporte les changements nécessaires pour que le mode protégé commence à être vraiment utilisé :

- La plus grande partie des enregistrements passent à 32 bits ;
- Il ajoute le mode protégé mais à 32 bits. Il est possible l'adressage de jusqu'à 4 Go de mémoire ;
- Les programmes sont encore divisés en segments mais ces segments peuvent avoir jusqu'à 4 Go ;
- L'augmentation de la taille des enregistrements se fait en combinant deux enregistrements utilisables séparément, par exemple, AX à 16 bits est composé d'AH (A High) et AL (A Low).

Attention : cette manière d'utiliser les enregistrements peut poser des problèmes au début pour les développeurs.

Le mode protégé à 32 bits

Il y a des différences par rapport au mode protégé en 16 bits du **processeur Intel 386** :

- Les déplacements sont étendus à 32 bits. Comme les déplacements peuvent aller jusqu'à 4 milliards, les segments peuvent avoir une taille de jusqu'à 4 Go ;
- Les segments peuvent être divisés dans des unités plus petites (de 4 Ko) appelées pages. C'est le principe de mémoire virtuelle permettant de maintenir uniquement certaines parties d'un segment en mémoire à un moment donné.
 - En 16 bits, le segment entier devait être en mémoire.
 - Cela n'est plus le cas avec les segments dans ce mode à 32 bits.

Le processeur Intel 80x486 et les Pentiums

Les processeurs suivants n'apporteront que des améliorations dans la performance mais très peu de nouvelles fonctionnalités. Ce sont :

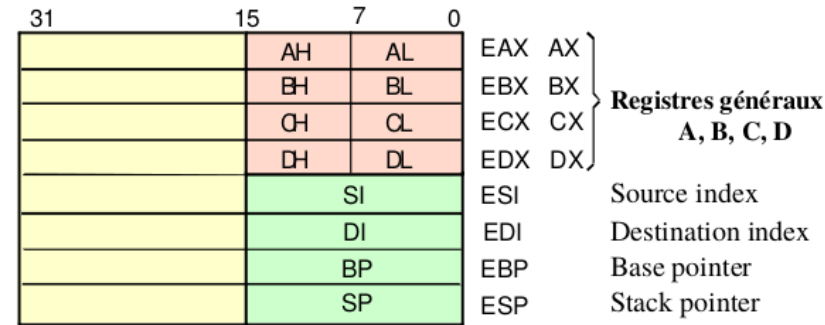
- **80486, Pentium et Pentium Pro** : Se sont des processeurs essentiellement plus rapides ;
- **Pentium MMX** : Ajoute les instructions Multi Média eXtensions (MMX) qui peuvent accélérer des opérations graphiques courantes ;
- **Pentium II** : C'est un Pentium Pro avec les instructions MMX ;
- **Pentium III** : C'est un Pentium II plus rapide;
- **Famille Core**: i3, i5, i7, i9 combinant des processeurs à plusieurs cœurs (jusqu'à 24).

Les enregistrements d'entiers dans les Pentiums 80x86

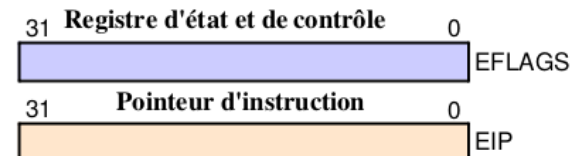
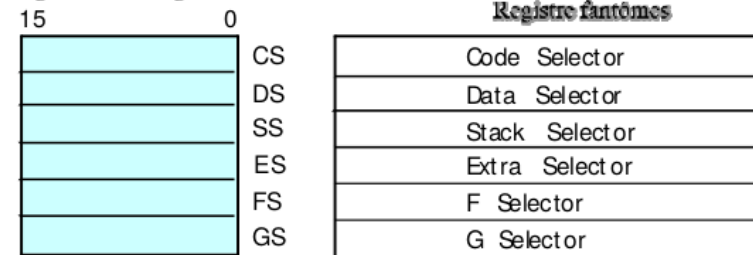
La structure de base des enregistrements dans des processeurs postérieurs au Pentium est la suivante :

C'est enregistrements permettent de gérer des entiers.

Certaines caractéristiques ont été conservées et d'autres sont moins utilisées.



Registres de Segment

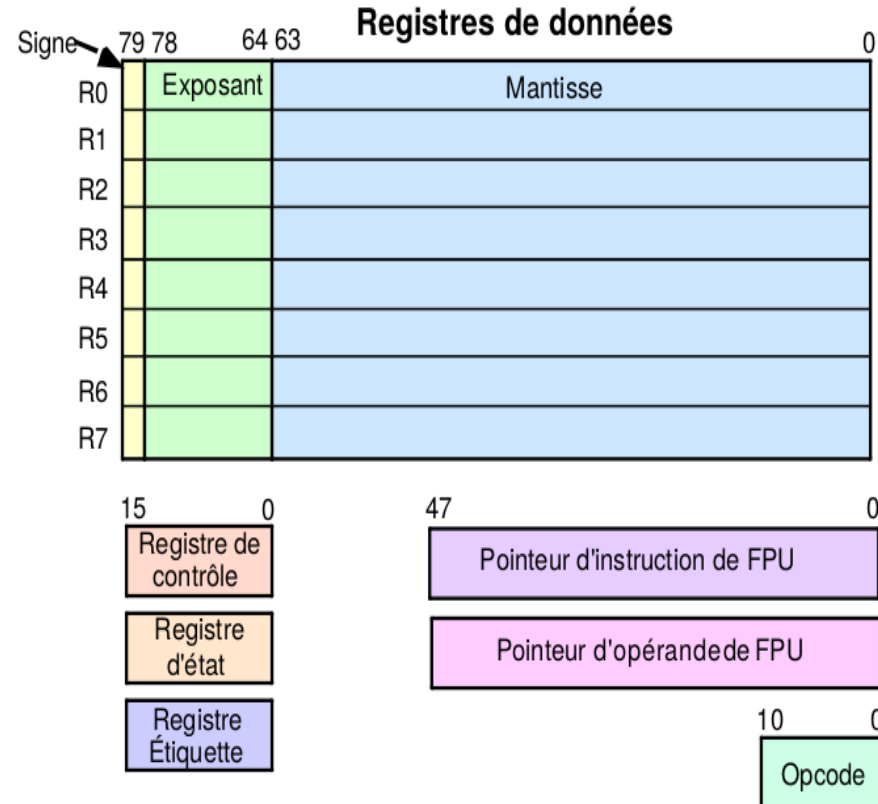


Les enregistrements des réels dans l'architecture Pentium 80x86

La structure de base des enregistrements pour les nombres à virgule flottante dans un Pentium est la suivante :

C'est enregistrements sont gérés comme une pile et tous ne sont pas accessibles directement.

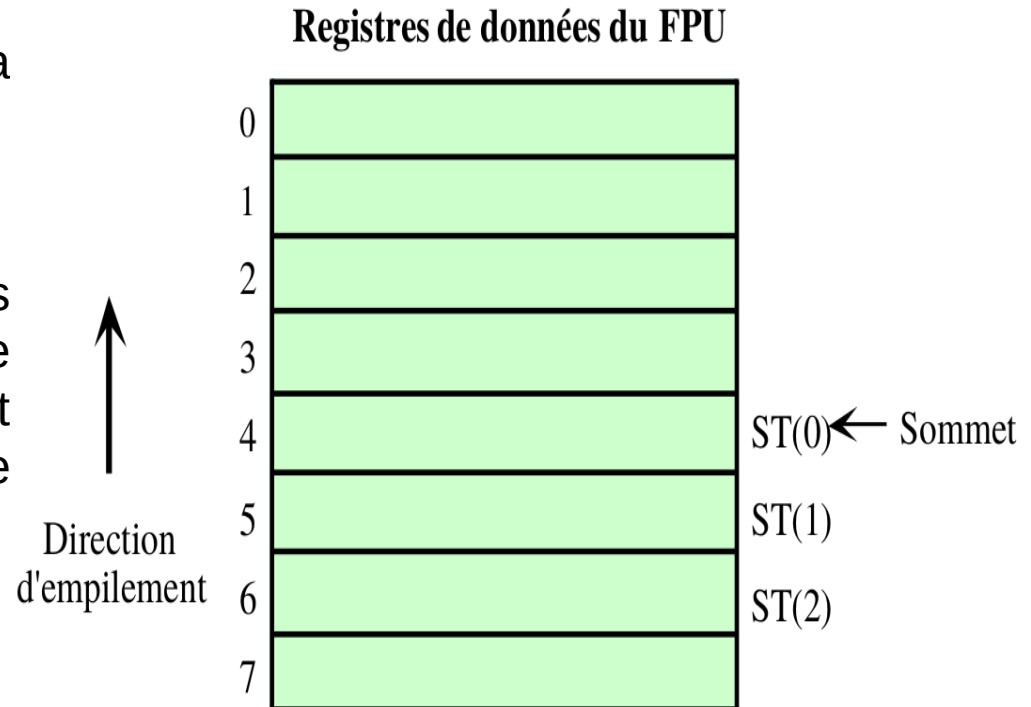
Ils ne peuvent pas être appelés directement par leur noms mais ils sont utilisés comme s'ils se trouvaient dans une pile (nous verrons cela en détail plus tard).



Les enregistrements des réels dans l'architecture Pentium 80x86

La direction empilement se fait de la position 7 à la position 0.

L'enregistrement ST0 sera toujours le plus au-dessus de la pile, si nous ajoutons une donnée, les enregistrements qui étaient avant deviendront ST1, ST2 et ainsi de suite.



Les interruptions

Elles servent à interrompre le fil d'exécution ordinaire d'un programme pour traiter des événements ayant besoin d'une réaction rapide.

C'est le **mécanisme d'interruption incorporé dans le matériel** qui permet de gérer ces événements.

Chaque type d'interruption a un nombre entier qui lui est affecté.

Des exemples d'événements :

- Le déplacement de la souris ;
- L'attente de la saisie d'une donnée ;
- L'affichage d'information dans la sortie standard ;

Gestionnaires d'interruption : ce sont des routines qui traitent les interruptions.

Le **vecteur d'interruption**, au début de la mémoire physique, contient les adresses segmentées des différents gestionnaires d'interruptions. Le numéro d'interruption est un indice de ce vecteur.

Les types d'interruptions

Externes : elles sont déclenchées depuis l'extérieur du processeur par :

- Les périphériques d'entrée/sortie ;
 - Clavier ;
 - Horloge ;
 - Lecteur de disque.

Internes : elles sont déclenchées à l'intérieur du processeur :

- Des erreurs dans l'exécution (aussi appelées **traps**) ;
- Lancées explicitement par des programmes (**interruptions logicielles**).

Important : sauf dans les cas des traps, une fois le traitement de l'interruption fini, elles retournent le contrôle au programme interrompu en restaurant son contexte (les enregistrements).

L'architecture de la CPU x86 à 64 bits

La CPU dispose des espaces en mémoire pour manipuler des données, **les registres**.

Les registres peuvent appartenir à quatre catégories :

- 1) **General purpose** ;
- 2) **Special-purpose application-accessible** ;
- 3) **Segment** : ils ne sont pas très utilisés dans les systèmes d'exploitation modernes ;
- 4) **Special-purpose kernel-mode** : utilisés pour écrire des systèmes d'exploitation, des débogueurs et des outils du système.

Nous allons nous concentrer sur les registres d'utilisation générale (ou general purpose).

Les registres de « general-purpose » ou généraux

- Seize registres de 64-bits avec les noms suivants : RAX, RBX, RCX, RDX, RSI, RDI, RBP, RSP, R8, R9, R10, R11, R12, R13, R14 et R15 ;
- Seize registres de 32-bits : EAX, EBX, ECX, EDX, ESI, EDI, EBP, ESP, R8D, R9D, R10D, R11D, R12D, R13D, R14D et R15D ;
- Seize registres de 16-bits : AX, BX, CX, DX, SI, DI, BP, SP, R8W, R9W, R10W, R11W, R12W, R13W, R14W et R15W ;
- Vingt registres de 8-bits : AL, AH, BL, BH, CL, CH, DL, DH, SIL, BPL, SPL, R8B, R9B, R10B, R11B, R12B, R13B, R14B et R15B.

Attention : ces registres ne sont pas indépendants, la modification d'un registre peut entraîner la modification de la valeur d'au plus trois autres registres.

De plus, ils ont parfois des utilisations spécialisées qui ne permettent pas au programmeur de les utiliser dans n'importe quel contexte ou de manière interchangeable. Par exemple, RSP est le pointeur vers le haut de la pile.

Relation entre les registres généraux (1/2)

La relation entre les enregistrements généraux est affiché sur la table suivante :

Bits 0–63	Bits 0–31	Bits 0–15	Bits 8–15	Bits 0–7
RAX	EAX	AX	AH	AL
RBX	EBX	BX	BH	BL
RCX	ECX	CX	CH	CL
RDX	EDX	DX	DH	DL
RSI	ESI	SI		SIL
RDI	EDI	DI		DIL
RBP	EBP	BP		BPL
RSP	ESP	SP		SPL
R8	R8D	R8W		R8B

Source : The Art of 64-bit Assembly Language

Relation entre les registres généraux (2/2)

La relation entre les enregistrements généraux est affichée sur la table suivante :

Bits 0–63	Bits 0–31	Bits 0–15	Bits 8–15	Bits 0–7
R9	R9D	R9W		R9B
R10	R10D	R10W		R10B
R11	R11D	R11W		R11B
R12	R12D	R12W		R12B
R13	R13D	R13W		R13B
R14	R14D	R14W		R14B
R15	R15D	R15W		R15B

Source : The Art of 64-bit Assembly Language

Quelques registres spéciaux

- **RAX** : Accumulateur, contient la valeur de retour des fonctions ;
- **RCX** : Compteur de boucle ;
- **RDX** : Partie haute d'une capacité de 64 bits ;
- **RSI** : Adresse source pour déplacement ou comparaison ;
- **RDI** : Adresse destination pour déplacement ou comparaison ;
- **RSP** : Pointeur vers le haut de la pile (stack) ;
- **RBP** : Pointeur vers le bas de la pile (stack) ;
- **RIP** : Compteur d'instruction (compteur du programme).

Registres pour des nombres à virgule flottante

Dans les CPUs x86-64 existe une unité de point flottante (FPU) (ou coprocesseur) x87 implémentant huit registres pour stocker des nombres à virgule flottante.

Ces registres sont nommés de ST0 à ST7 et ils ont une capacité initiale de 80 bits.

Important : Ils sont stockés comme une pile. Ainsi, il n'est pas possible d'accéder directement qu'à celui en haut de la pile ou aux deux premiers de la pile.

Chacun de ces registres peut stocker une représentation en virgule flottante de double précision étendue. C'est à dire, 1 bit de signe, 15 bits d'exposant et 64 bits pour la mantisse.

NOTE : On passera très rapidement sur leur utilisation dans le deuxième CM.

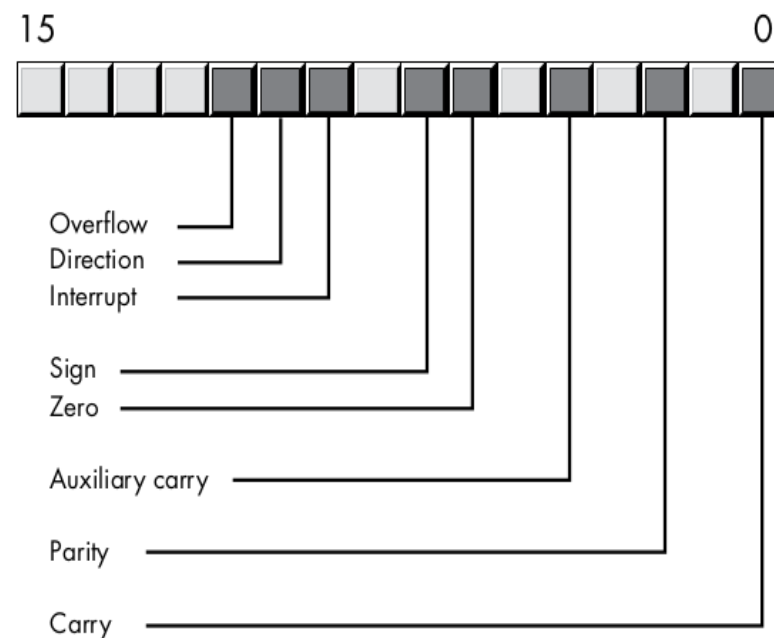
Les bits d'état ou flags

C'est un registre de 64-bits contenant un ensemble de valeurs à vrai ou faux.

Ces valeurs sont généralement utilisées par le mode Kernel et la programmation de systèmes d'exploitation.

Uniquement huit de ces valeurs sont, en général, pertinentes pour un programmeur en assembleur.

Important : les flags de codes de condition (overflow, carry, sign et zero) permettent de connaître le résultat des calculs, par exemple, une comparaison des nombres.



Le sous-système de mémoire

Il stocke et gère des données comme les variables, les constantes, les instructions des programmes, etc.

Cellule : c'est l'unité minimale de stockage des données. La combinaison des cellules peut permettre de structurer des ensembles de données plus larges.

Important : L'architecture x86-64 permet d'accéder aux données à niveau d'octet.

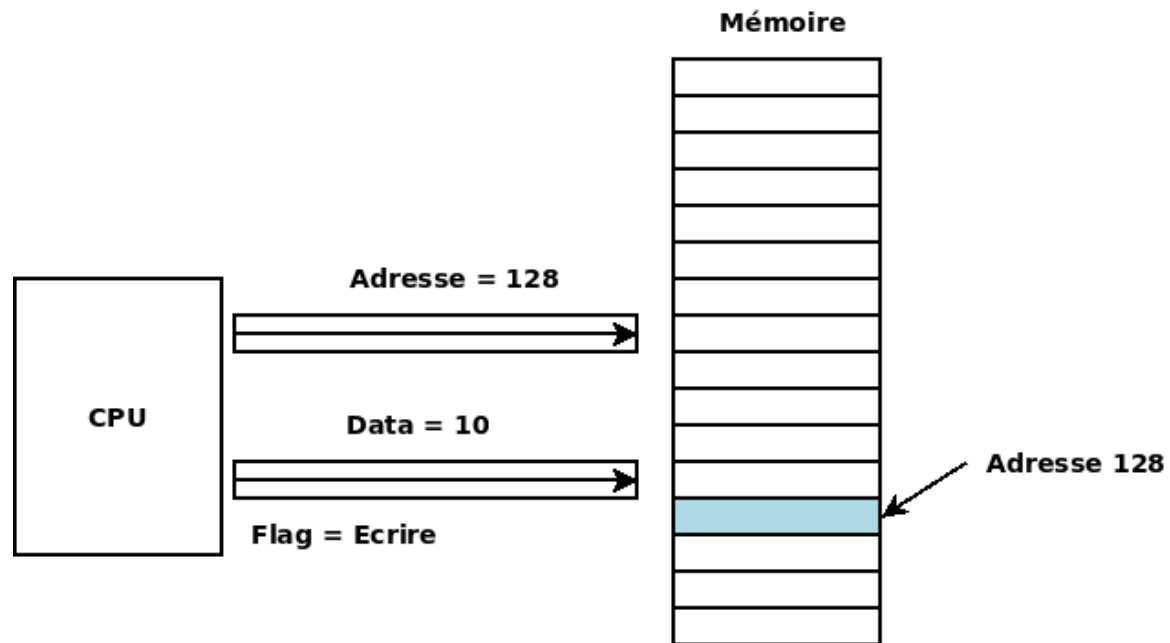
L'accès aux données est réalisé via l'adresse de la cellule en mémoire considérée comme un array où la première adresse est 0 et la dernière est $2^{32} - 1$ pour une mémoire de 4 Giga-octets.

Le sous-système de mémoire

Il existe un bus d'adressage et un bus de données entre la CPU et le système de mémoire.

Des flags permettent de signaler si la CPU est en train de lire ou d'écrire des données dans la mémoire.

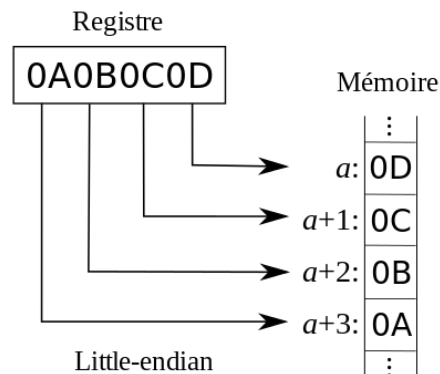
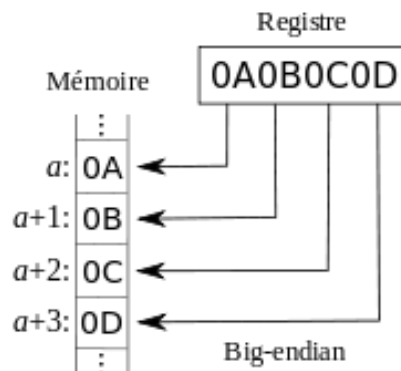
Pour des données plus grandes qu'un octet, des cellules consécutives en mémoire sont utilisées. Pour y accéder, il faut utiliser l'adresse la moins élevée.



Big endian et Little endian

Ce sont deux manières de structurer l'information en mémoire :

- **Big-endian** : les bits le plus significatifs sont dans les adresses les moins élevées. Utilisée par les processeurs Motorola 68000 et les SPARC de Sun Microsystems.
- **Little-endian** : les bits les plus significatifs sont dans les plus élevées. Utilisé par les processeurs Intel (ceux que nous allons utiliser).



Le stockage dans la mémoire dans l'architecture Intel x86

Little-endian

L'allocation de la mémoire se fait à partir de l'adresse la plus basse et vers le haut des adresses en mémoire :

Pour des données plus grandes qu'un octet, des cellules consécutives en mémoire sont utilisées. Pour y accéder, il faut utiliser l'adresse la moins élevée.

Important : Il est possible d'avancer explicitement sur les adresses en mémoire, assembleur permet de faire ces déplacements au niveau d'octet.

